Assignment 6 – Peripherals and Interfacing

Unit 6: Embedded Systems

Long Answer Questions (8 Marks Each)

Q1. Define sensor interfacing. Explain its importance in embedded systems with two real-world examples.

Answer:

Definition of Sensor Interfacing

Sensor interfacing refers to the process of connecting a sensor (a physical device that detects and responds to environmental stimuli) to a microcontroller or processor so that the sensor's output can be read, processed, and used to trigger appropriate responses. This connection involves both hardware (physical wiring, signal conditioning) and software (register configuration, data reading routines) components.

In embedded systems, the microcontroller cannot directly understand physical quantities like temperature, pressure, humidity, or light. Sensors convert these physical quantities into electrical signals (voltage or current), and the interfacing process ensures these signals are correctly interpreted by the microcontroller.

Importance of Sensor Interfacing in Embedded Systems

- **Environmental Awareness:** Sensors are the primary means by which an embedded system perceives its environment. Without proper interfacing, the system would be blind to real-world changes.
- **Real-Time Decision Making:** Properly interfaced sensors enable the microcontroller to make real-time decisions. For example, detecting a temperature rise and immediately activating a cooling fan.
- **System Automation:** Sensor interfacing is the backbone of automation. Systems like smart home controllers, industrial automation, and medical devices all depend on accurate sensor data.
- **Data Accuracy:** Proper interfacing ensures that the signal from the sensor reaches the microcontroller without loss of accuracy—using ADC for analog sensors, pull-up/down resistors, and shielding against noise.
- **Safety and Reliability:** In safety-critical systems (medical devices, automobiles), accurate sensor interfacing is essential to prevent system failures.

Real-World Example 1: Smart Temperature Control System

In a smart air conditioning system, a temperature sensor (e.g., LM35) is interfaced with a microcontroller. The LM35 outputs 10 mV per degree Celsius. The analog voltage is fed into the ADC channel of the ATmega32. The microcontroller reads the ADC value, converts it to temperature using the formula (ADC_value / 4), and compares it with a set threshold. If the temperature exceeds the

Available on notes.arpanadhikari7.com.np

threshold, the microcontroller activates a relay or PWM-driven fan motor to cool the room. The LCD displays the current temperature. This entire system depends on correct sensor interfacing.

Real-World Example 2: Automatic Door System Using PIR Sensor

In an automatic door system, a PIR (Passive Infrared) motion sensor is interfaced with a microcontroller. The PIR sensor detects infrared radiation emitted by humans. When motion is detected, the sensor output goes HIGH (logic 1), which is read by the microcontroller via a GPIO pin configured as input. The microcontroller then signals a motor driver IC (L293D) to rotate a DC motor and open the door. After a set delay, the door is closed again. This application demonstrates the importance of correct sensor interfacing for real-world automation.

Q2. Differentiate between analog sensors and digital sensors. Compare them based on: Output type, Accuracy, Interfacing complexity, Applications.

Answer:

Sensors can broadly be classified into two types based on the nature of their output signal: Analog Sensors and Digital Sensors. Understanding the differences is essential for selecting the right sensor for an application.

a) Output Type

- **Analog Sensors:** Produce a continuous voltage or current output that varies proportionally with the measured physical quantity. Example: LM35 outputs 10 mV/°C — at 25°C it outputs 250 mV, a continuously varying signal.
- **Digital Sensors:** Produce a discrete binary output — either HIGH (1) or LOW (0), or a digital serial data stream (I2C, SPI, 1-Wire). Example: DHT11 outputs temperature and humidity as a serial digital code.

b) Accuracy

- **Analog Sensors:** Accuracy depends on the quality of the ADC used to convert the signal. Noise, reference voltage instability, and ADC resolution all affect accuracy. An 8-bit ADC gives 256 levels; a 10-bit ADC gives 1024 levels. They are susceptible to noise.
- **Digital Sensors:** Generally more accurate and immune to electrical noise since the signal is in binary form. They include internal calibration and error correction. DHT11 has ±2°C accuracy; more advanced digital sensors like SHT31 have ±0.2°C.

notes.arpanadhikari7.com.np

c) Interfacing Complexity

- **Analog Sensors:** Require an ADC (internal or external) to convert the analog output. Need voltage regulators, decoupling capacitors, and sometimes op-amp signal conditioning. More hardware components are needed.
- **Digital Sensors:** Communicate via standard protocols (I2C, SPI, UART, 1-Wire). Require only a few wires and protocol libraries. However, software complexity increases due to communication protocols.

d) Applications

- **Analog Sensors:** Used in simple, cost-sensitive applications: LM35 for basic temperature measurement, LDR for light sensing, potentiometers, pressure transducers in industrial systems.

Available on notes.arpanadhikari7.com.np

- **Digital Sensors:** Used in modern, precision-required systems: DHT11/DHT22 for weather stations, MPU6050 (gyroscope/accelerometer) in drones, BMP280 (pressure) in GPS devices, and DS18B20 for medical thermometers.

Q3. Explain the working principle of a temperature sensor (e.g., LM35 or DHT11). Draw a neat block diagram showing its interfacing with a microcontroller.

Answer:

LM35 Temperature Sensor – Working Principle

The LM35 is a precision analog temperature sensor manufactured by Texas Instruments. It is designed to provide an output voltage linearly proportional to the Celsius temperature scale.

Key Characteristics

- Output voltage: 10 mV per degree Celsius (e.g., 250 mV at 25°C)
- Operating temperature range: -55°C to +150°C
- Supply voltage: 4V to 30V DC
- Accuracy: ±0.5°C at room temperature
- Low self-heating: less than 0.1°C in still air
- Package: TO-92 (3-pin), TO-220

Internal Working Principle

The LM35 uses a temperature-sensitive transistor circuit based on the principle that the base-emitter voltage (Vbe) of a transistor decreases linearly with increasing temperature. The internal circuit amplifies this voltage change and provides a scaled output of exactly 10 mV/°C. The device internally compensates for non-linearities to maintain accuracy across the full temperature range.

Pin Configuration

- Pin 1 (Vcc): Power supply (5V)
- Pin 2 (Vout): Analog output proportional to temperature
- Pin 3 (GND): Ground

Interfacing with ATmega32 Microcontroller

To interface LM35 with ATmega32: Vcc → 5V; GND → GND; Vout → PA0 (ADC Channel 0). The ATmega32's internal 10-bit ADC reads the analog voltage. With Vref = 2.56V (internal), step size = 2.56/1024 = 2.5 mV/step. Since LM35 outputs 10 mV/°C = 4 ADC steps per degree, temperature = ADC_value / 4 in Celsius.

Block Diagram Description

[LM35 Sensor] → [Analog Voltage Output (10mV/°C)] → [ADC Channel (PA0 of ATmega32)] → [CPU Processing: T = ADC/4] → [LCD Display / Control Output]

The block diagram shows: LM35 connected to the ADC input; the ADC converts analog voltage to 10-bit digital value; the CPU reads ADCL/ADCH registers; temperature is calculated; the result is displayed on LCD or used for control.

DHT11 Temperature and Humidity Sensor

The DHT11 is a digital sensor that measures both temperature (0–50°C, ±2°C accuracy) and humidity (20–90% RH, ±5% accuracy). It uses a single-wire serial protocol. The sensor contains a resistive

Available on notes.arpanadhikari7.com.np

humidity component and an NTC thermistor. After a start signal from the microcontroller, it sends 40 bits of serial data (16 bits humidity + 16 bits temperature + 8 bits checksum).

Q4. What is an ADC (Analog-to-Digital Converter)? Explain its role in interfacing analog sensors with microcontrollers.

Answer:

Definition of ADC

An Analog-to-Digital Converter (ADC) is an electronic circuit that converts a continuous analog signal (voltage or current) into a discrete digital number that can be processed by a digital system such as a microcontroller. Since microcontrollers operate on binary data, an ADC acts as the critical bridge between the analog physical world and the digital processing world.

Need for ADC in Embedded Systems

Physical quantities such as temperature, pressure, humidity, and light are inherently analog — they vary continuously over time. Sensors convert these quantities into proportional electrical voltages. However, microcontrollers can only process digital binary values (0 or 1). The ADC performs the quantization — mapping a range of analog voltages to discrete binary codes.

Key Characteristics of ADC

- **Resolution:** Defines the number of discrete levels the ADC can produce. An n-bit ADC produces 2^n levels. ATmega32 has a 10-bit ADC (1024 levels). Formula: Step Size = Vref / 2^n
- **Reference Voltage (Vref):** The maximum measurable voltage. ATmega32 supports AVCC (5V), External AREF pin, or Internal 2.56V reference.
- **Conversion Time:** Time taken to complete one conversion. ATmega32 takes 13 ADC clock cycles. At CK/128 with 8 MHz clock: ADC clock = 62.5 kHz; conversion time ≈ 208 µs.
- **Digital Output:** Vin = Dout × Step Size. For 10-bit ADC with Vref = 2.56V: if temperature is 25°C, LM35 outputs 250 mV → ADC output = 250/2.5 = 100.

ADC in ATmega32 – Key Features

- 10-bit resolution (1024 steps)
- 8 analog input channels (PA0–PA7)
- Result stored in ADCL (low byte) and ADCH (high byte) registers
- ADMUX register: selects reference voltage and input channel
- ADCSRA register: controls enable (ADEN), start conversion (ADSC), prescaler (ADPS)

Role of ADC in Sensor Interfacing

1. Signal Acquisition: The sensor output (e.g., 0–2.5V from LM35) is connected to an ADC input channel.
2. Voltage to Digital Conversion: The ADC samples the analog voltage and produces a digital code proportional to it.
3. Data Processing: The microcontroller reads ADCL/ADCH and applies conversion formulas (e.g., T = ADC/4 for LM35 with 2.56V Vref).
4. Control Output: Processed data is used to control actuators, display readings, or trigger alarms.

Programming Steps for ADC in ATmega32

5. Set DDRA = 0x00 (Port A as input)
6. Set ADMUX = 0xC0 (Vref = 2.56V internal, ADC0 channel, right-justified)

Available on notes.arpanadhikari7.com.np

7. Set ADCSRA = 0x87 (Enable ADC, prescaler = CK/128)
8. Set ADSC bit to start conversion
9. Wait for ADIF flag to set (conversion complete)
10. Read ADCL first, then ADCH to get full 10-bit result

Q5. Describe the working and interfacing of a humidity sensor. How is humidity data represented and processed?

Answer:

Introduction to Humidity Sensing

Humidity refers to the amount of water vapor present in the air. Relative Humidity (RH) is expressed as a percentage — the ratio of actual water vapor pressure to the saturation vapor pressure at a given temperature. Humidity sensors (hygrometers) measure this parameter and are essential in weather monitoring, HVAC systems, agricultural automation, and industrial process control.

DHT11 Humidity Sensor – Working Principle

The DHT11 is one of the most widely used low-cost sensors that measures both temperature (0–50°C) and relative humidity (20–90% RH). It contains two sensing elements: a resistive humidity sensor (NTC type) and a NTC thermistor for temperature. These elements are connected to an internal 8-bit microcontroller that calibrates and digitizes the readings.

Internal Components

- Resistive humidity sensing element: Its electrical resistance changes with ambient humidity
- NTC (Negative Temperature Coefficient) thermistor: Resistance decreases as temperature increases
- Internal calibration IC: Processes raw sensor data and applies stored calibration coefficients

DHT11 Communication Protocol (Single-Wire)

DHT11 uses a proprietary single-wire serial protocol. The communication sequence is:

11. MCU sends start signal: pulls data line LOW for at least 18 ms, then HIGH for 20–40 µs
12. DHT11 responds: pulls data line LOW for 80 µs, then HIGH for 80 µs
13. DHT11 transmits 40 bits: 8 bits humidity integer + 8 bits humidity decimal + 8 bits temperature integer + 8 bits temperature decimal + 8 bits checksum
14. Bit encoding: '0' = 50µs LOW + 26–28µs HIGH; '1' = 50µs LOW + 70µs HIGH

Data Representation

The 40-bit data packet format: [8-bit RH integer][8-bit RH decimal][8-bit T integer][8-bit T decimal][8-bit checksum]. Example: If data = 0x32 00 18 00 4A → Humidity = 50% RH, Temperature = 24°C. Checksum = 0x32+0x00+0x18+0x00 = 0x4A (valid).

Interfacing DHT11 with ATmega32

- Vcc → 5V; GND → GND; Data → PD0 (GPIO pin)
- A 4.7kΩ pull-up resistor is connected between Data and Vcc
- PD0 is configured as output during start signal, then switched to input for data reception

Data Processing

After receiving the 40 bits, the MCU: (1) Separates the 5 bytes; (2) Validates using checksum (Byte1+Byte2+Byte3+Byte4 should equal Byte5); (3) Extracts humidity value from Byte1 (integer part)

Available on notes.arpanadhikari7.com.np

and Byte2 (decimal); (4) Extracts temperature from Byte3 and Byte4; (5) Displays values on LCD or transmits via UART.

Q6. Explain the operation of a motion sensor (PIR). List at least four applications of motion sensors in embedded systems.

Answer:

PIR (Passive Infrared) Sensor – Introduction

A PIR (Passive Infrared) sensor detects infrared radiation emitted by warm objects, particularly humans and animals. The sensor is called 'passive' because it does not emit any radiation itself — it only detects existing infrared energy. PIR sensors are widely used in motion detection applications due to their low cost, low power consumption, and reliability.

Construction of PIR Sensor

- **Pyroelectric Element:** The core sensing element is a pyroelectric crystal that generates an electric charge when exposed to infrared radiation. Most PIR sensors use two such elements connected in opposite polarity.
- **Fresnel Lens:** A specially shaped plastic lens that focuses and directs the infrared radiation onto the pyroelectric element. It divides the detection area into multiple zones.
- **Signal Processing IC:** Amplifies the weak pyroelectric signal, filters out low-frequency noise, and provides a clean digital output.
- **HC-SR501 Module:** A commonly used PIR module with adjustable sensitivity and time-delay potentiometers, and a retriggering jumper.

Working Principle

All objects above absolute zero temperature emit infrared radiation. The human body emits IR at approximately 9.4 micrometers wavelength. The PIR sensor's two pyroelectric elements are arranged so that when there is no movement, both elements receive the same IR level and cancel each other out — the output remains LOW.

When a person moves across the sensor's field of view, the Fresnel lens alternately focuses IR onto one element and then the other. This creates a differential signal between the two elements. The signal processing IC amplifies this differential and outputs a HIGH signal to the microcontroller.

notes.arpanadhikari7.com.np

Interfacing PIR with Microcontroller

- Vcc → 5V (HC-SR501 works at 5–12V); GND → GND; Output → GPIO input pin (e.g., PD2/INT0)
- Output is HIGH (5V) when motion detected, LOW (0V) when idle
- Detection range: up to 7 meters; Detection angle: approximately 110°
- Time delay and sensitivity can be adjusted using onboard potentiometers

Four Applications of Motion Sensors in Embedded Systems

- **Application 1 – Automatic Door System:** A PIR sensor is placed at the entrance. When a person approaches, the sensor output goes HIGH, and the microcontroller drives a DC motor via an H-bridge (L293D) to open the door automatically. The door closes after a set delay.
- **Application 2 – Home Security Alarm:** PIR sensors are installed in rooms. On detecting unauthorized movement, the microcontroller activates a buzzer and/or sends an SMS alert via a GSM module. The LCD displays a warning message.

Available on notes.arpanadhikari7.com.np

- **Application 3 – Smart Street Lighting:** Street lights equipped with PIR sensors activate only when pedestrians or vehicles are detected, saving electricity. PWM-controlled LEDs dim automatically when no motion is detected.
- **Application 4 – Energy-Saving Office Lighting:** Lights and air conditioning in offices are controlled based on PIR motion detection. If no person is detected for a set time, the system shuts off electrical appliances automatically.

Q7. What is an actuator? Explain the difference between sensors and actuators with examples.

Answer:

Definition of Actuator

An actuator is an electromechanical device that converts electrical energy into physical motion, force, or action. It acts as the output component of an embedded system — executing commands given by the microcontroller to interact with the physical world. Actuators are the 'muscles' of an embedded system, just as sensors are its 'senses'.

Types of Actuators

- **DC Motors:** Convert electrical energy into continuous rotational motion. Speed controlled by PWM.
- **Stepper Motors:** Move in discrete angular steps for precise positioning. Used in CNC, 3D printers.
- **Servo Motors:** Closed-loop motors for precise angular positioning. Used in robotics.
- **Solenoids:** Convert electrical current into linear mechanical movement. Used in locks and valves.
- **Buzzers/Speakers:** Convert electrical signals into sound. Used in alarms and audio systems.
- **Relays:** Electrically controlled switches that connect/disconnect high-power circuits.

Difference between Sensors and Actuators

- **Role:** Sensor = Input device (collects environmental data). Actuator = Output device (performs physical action based on processed data).
- **Direction of Operation:** Sensor: Physical world → Electrical signal. Actuator: Electrical signal → Physical world.
- **Signal Type:** Sensors provide input signals (voltage, current, digital pulses) to the MCU. Actuators receive output signals from the MCU.
- **Examples of Sensors:** LM35 (temperature), PIR (motion), LDR (light), DHT11 (humidity), ultrasonic sensor (distance).
- **Examples of Actuators:** DC motor (rotation), servo motor (position), buzzer (sound), LCD (display), relay (switching).

System Flow Example: Smart Temperature Control

Sensor Stage: LM35 temperature sensor detects ambient temperature → converts it to analog voltage (e.g., 250 mV at 25°C). Processing Stage: ATmega32 ADC converts the voltage to digital value → CPU compares with threshold. Actuator Stage: If temperature > 30°C, the microcontroller sends PWM signal to a DC fan motor (actuator) to increase speed. The LCD (another actuator) displays the current temperature and fan status.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Q8. Compare DC motor, Stepper motor, and Servo motor based on: Control method, Precision, Applications, Advantages and limitations.

Answer:

Overview

DC motors, stepper motors, and servo motors are three primary actuators used in embedded systems. Each has distinct operating principles, control methods, and application domains. The choice depends on requirements such as speed, precision, torque, and cost.

a) Control Method

- **DC Motor:** Controlled by applying a DC voltage across its terminals. Speed is varied by changing the applied voltage (using PWM). Direction is controlled by reversing polarity (H-bridge circuit). No feedback required — open-loop control.
- **Stepper Motor:** Controlled by sequentially energizing coils in a specific pattern using digital pulses. Each pulse advances the motor by one step (e.g., 1.8° per step for 200-step motor). Uses wave drive, full-step, half-step, or micro-step modes. Open-loop control.
- **Servo Motor:** Controlled by PWM signals. Pulse width (1–2 ms repeated every 20 ms) determines the shaft angle (0°–180°). Uses closed-loop control with a potentiometer or encoder as feedback to maintain the desired position.

b) Precision

- **DC Motor:** Lowest precision among the three. Speed can vary with load. Position control requires external encoders/feedback.
- **Stepper Motor:** High positional precision in open-loop mode. Typical step angle is 1.8° (200 steps/revolution). Can miss steps under overload. Micro-stepping improves resolution.
- **Servo Motor:** Highest precision. Closed-loop feedback continuously corrects position error. Suitable for applications requiring exact angular positioning.

c) Applications

- **DC Motor:** Cooling fans, electric vehicles, conveyor belts, toys, computer disk drives, drilling machines.
- **Stepper Motor:** CNC machines, 3D printers, plotters, camera platforms, robotic joints, hard disk drives.
- **Servo Motor:** Robotic arms, RC vehicles, camera gimbals, industrial automation, prosthetic limbs.

notes.arpanadhikari7.com.np

d) Advantages and Limitations

- **DC Motor – Advantages:** Simple, inexpensive, high speed, continuous rotation, easy to control.
- **DC Motor – Limitations:** Poor precision, brushes wear out (brushed DC), speed varies with load.
- **Stepper Motor – Advantages:** Excellent open-loop positioning, no feedback required, precise step control.
- **Stepper Motor – Limitations:** Can miss steps at high loads, heat generation at standstill, lower efficiency.
- **Servo Motor – Advantages:** High precision, closed-loop stability, handles varying loads, compact.

Available on notes.arpanadhikari7.com.np

- **Servo Motor – Limitations:** More expensive, complex control circuit, limited rotation range (typically 0°–180° for hobby servos).

Q9. Describe the working principle of a stepper motor. Explain full-step and half-step operation.

Answer:

Stepper Motor – Introduction and Working Principle

A stepper motor is an electromechanical device that converts electrical pulses into precise mechanical rotation. Unlike DC motors that spin continuously, stepper motors move in fixed angular increments called 'steps'. This makes them ideal for applications requiring precise positioning without a position feedback sensor.

The motor consists of two main parts: the stator (stationary part with multiple coils) and the rotor (rotating part made of permanent magnets or ferromagnetic material). When a coil on the stator is energized, it creates a magnetic field that attracts the rotor's magnetic poles, causing the rotor to align with that field. By energizing coils in sequence, a rotating magnetic field is created, and the rotor follows step by step.

Step Angle Formula

Step Angle (θ) = 360° / (N × P), where N = number of teeth on the rotor, P = number of phases. A common stepper motor has a step angle of 1.8° (200 steps per revolution).

Full-Step Operation (Two-Phase Excitation)

In full-step operation, two coils are energized simultaneously. This provides maximum torque as both coils contribute to the magnetic field. The rotor completes one full revolution in the same number of steps as the motor's tooth/pole count.

Coil energization sequence for full-step (4 steps per cycle, CW rotation):

- Step 1: Winding A=1, B=1, C=0, D=0
- Step 2: Winding A=0, B=1, C=1, D=0
- Step 3: Winding A=0, B=0, C=1, D=1
- Step 4: Winding A=1, B=0, C=0, D=1

Advantages of Full-Step: Maximum torque, simpler control logic, widely used for industrial applications. Limitation: Relatively large step angle, less smooth rotation.

Half-Step Operation

Half-step operation alternates between energizing one coil and two coils simultaneously. This effectively doubles the number of steps per revolution, reducing the step angle by half (e.g., from 1.8° to 0.9° for an 8-step cycle).

Half-step sequence (8 steps per cycle):

- Step 1: A=1, B=0, C=0, D=0
- Step 2: A=1, B=1, C=0, D=0
- Step 3: A=0, B=1, C=0, D=0
- Step 4: A=0, B=1, C=1, D=0
- Step 5: A=0, B=0, C=1, D=0
- Step 6: A=0, B=0, C=1, D=1
- Step 7: A=0, B=0, C=0, D=1

Available on notes.arpanadhikari7.com.np

- Step 8: A=1, B=0, C=0, D=1

Advantages of Half-Step: Higher resolution (smoother motion), finer positioning accuracy. Limitation: Torque varies between single and two-coil excitation states, slightly complex control.

Wave Drive (Single-Step Mode) for Comparison

In wave drive mode, only one coil is energized at a time. This reduces power consumption but provides less torque (approximately 70% of full-step). It is suitable for light-load applications.

Q10. Explain servo motor control using PWM signals. Why is PWM necessary for servo motors?

Answer:

Servo Motor – Introduction

A servo motor is a rotary actuator designed for precise angular position control. It uses a closed-loop feedback mechanism to maintain its shaft at a commanded angle. A standard hobby servo motor typically rotates between 0° and 180°. Industrial servo motors can achieve continuous 360° rotation.

Components of a Servo Motor System

- **DC Motor:** Provides rotational energy
- **Gear Train:** Reduces speed and increases torque
- **Potentiometer:** Provides position feedback (angular position as voltage)
- **Control Circuit:** Comparator that compares desired position (PWM) with actual position (potentiometer) and drives the motor to minimize error

PWM Control of Servo Motors

A servo motor is controlled by sending it a PWM (Pulse Width Modulation) signal with a fixed frequency of 50 Hz (period = 20 ms). The width (duration) of the HIGH pulse within this 20 ms window determines the angular position of the shaft.

Pulse Width to Position Mapping

- Pulse width = 1.0 ms → Position = 0° (minimum)
- Pulse width = 1.5 ms → Position = 90° (neutral/center)
- Pulse width = 2.0 ms → Position = 180° (maximum)
- For intermediate angles: Pulse width = 1.0 ms + (angle/180°) × 1.0 ms

notes.arpanadhikari7.com.np

Why PWM is Necessary for Servo Motors

- **Position Encoding:** The servo motor's internal comparator needs a reference signal to determine the desired position. PWM pulse width encodes the position information uniquely — no other simple signal type can carry this information continuously.
- **Continuous Feedback Comparison:** The servo's control circuit continuously compares the incoming PWM pulse width with the potentiometer voltage. PWM must be repeated every 20 ms (50 Hz) to keep the servo locked in position. Without PWM, the servo loses its position hold.
- **Efficient Power Control:** PWM allows the servo to receive power information without large analog circuits. The digital nature of PWM makes it noise-immune and easy to generate from microcontroller timers.
- **Closed-Loop Stability:** The repetitive PWM signal provides a continuous reference point. If an external force disturbs the servo, the control circuit detects the error (difference between PWM command and potentiometer feedback) and corrects it immediately.

Available on notes.arpanadhikari7.com.np

Generating PWM for Servo on ATmega32

Method 1 (Software delay): Set output pin HIGH → delay_us(pulse_width) → set LOW → delay_ms(20 - pulse_width_ms). Method 2 (Hardware Timer): Configure Timer1 in Fast PWM mode 14. Set ICR1 = 19999 (20ms period at 1MHz), OCR1A = position value (1000–2000 for 1–2 ms).

Q11. Define Pulse Width Modulation (PWM). Explain its working principle with suitable waveform.

Answer:

Definition of PWM

Pulse Width Modulation (PWM) is a technique used to encode information or control power delivery by varying the width (duration) of the HIGH portion of a digital pulse signal while keeping the total period (frequency) constant. The ratio of the ON time to the total period is called the duty cycle, expressed as a percentage.

Key Parameters of PWM

- **Period (T):** The total time for one complete cycle of the PWM signal (ON + OFF time). Period = 1/Frequency.
- **Pulse Width (TON):** The duration for which the signal remains HIGH in one cycle.
- **Duty Cycle (D):** D = (TON / T) × 100%. A 50% duty cycle means the signal is HIGH for half the period.
- **Frequency:** Number of PWM cycles per second. Common values: 50 Hz (servo control), 1 kHz–20 kHz (motor speed control), 100 Hz–1 kHz (LED dimming).

Working Principle

A PWM signal switches between HIGH (Vcc) and LOW (GND) at a fixed frequency. By varying the duty cycle: at 0% duty cycle, the output is always LOW (no power); at 100% duty cycle, the output is always HIGH (full power); at 50% duty cycle, the output is HIGH for half the period (50% average power).

The average voltage of a PWM signal = Vcc × Duty Cycle. Example: With Vcc = 5V and 60% duty cycle, average voltage = 3V. This allows analog-equivalent control using digital signals.

PWM Waveform Description

25% Duty Cycle: ┐┌┐┌┐┌ (SHORT pulses) → Low average voltage → Slow motor / Dim LED

50% Duty Cycle: ┐└┐└┐└ (Equal ON/OFF) → Medium average voltage

75% Duty Cycle: ────┐└──── (LONG pulses) → High average voltage → Fast motor / Bright LED

PWM Generation in ATmega32

ATmega32 has three timers (Timer0, Timer1, Timer2) capable of generating PWM. Timer1 is 16-bit and supports Fast PWM, Phase Correct PWM, and CTC modes. OCR1A/OCR1B registers set the duty cycle. TCCR1A/TCCR1B registers configure mode and prescaler.

Q12. Explain how PWM can be used to: a) Control LED brightness, b) Control DC motor speed.

Answer:

Available on notes.arpanadhikari7.com.np

a) LED Brightness Control Using PWM

An LED's brightness is directly proportional to the average current flowing through it. By varying the duty cycle of a PWM signal, we can control the average current and thus the perceived brightness of the LED.

The human eye perceives flicker at frequencies below ~50 Hz. PWM frequencies above 100 Hz make the LED appear to glow steadily at varying brightness levels. The persistence of vision effect averages the rapid ON/OFF pulses.

Working: The LED is connected to the PWM output pin of the microcontroller (e.g., OCR1A on ATmega32). By changing the OCR1A value, the duty cycle changes:

- OCR1A = 0: 0% duty cycle → LED OFF
- OCR1A = ICR1/2: 50% duty cycle → LED at half brightness
- OCR1A = ICR1: 100% duty cycle → LED at full brightness

Application: RGB LED dimmers, display backlighting, theater stage lighting, sunrise alarm clocks (gradually increasing LED brightness to simulate sunrise).

b) DC Motor Speed Control Using PWM

A DC motor's speed is proportional to the average voltage applied to its terminals. PWM provides an efficient way to control this average voltage without wasting energy as heat (unlike resistive voltage dividers).

Working: The PWM signal is applied to the control input of a motor driver IC (e.g., L293D or L298N). The driver IC provides the actual motor current — the microcontroller's PWM output is only a control signal.

- 0% duty cycle: Motor OFF (no voltage)
- 25% duty cycle: Motor runs at ~25% speed
- 50% duty cycle: Motor runs at ~50% speed
- 75% duty cycle: Motor runs at ~75% speed
- 100% duty cycle: Motor runs at full speed

The L293D/L298N receives: EN (Enable) pin connected to PWM output for speed control; IN1, IN2 pins for direction control (H-bridge switching). By combining PWM speed control with H-bridge direction control, complete motor control is achieved.

Advantages of PWM for motor control: High efficiency (no power wasted in resistors); smooth speed variation; digital control from MCU; easy integration with feedback systems for closed-loop control.

notes.arpanadhikari7.com.np

Q13. Explain the working principle of an LCD (16×2). What are the functions of the RS, RW, and EN pins?

Answer:

Introduction to LCD 16×2

The LCD 16×2 (Liquid Crystal Display) is one of the most widely used display modules in embedded systems. It can display 16 characters per row across 2 rows, giving a total of 32 characters simultaneously. It is based on the HD44780 controller IC, which simplifies communication with microcontrollers.

Working Principle of LCD

Available on notes.arpanadhikari7.com.np

Liquid crystals are materials that can change their optical properties when an electric field is applied. The LCD consists of two glass plates with a thin layer of liquid crystal molecules between them. Polarizing filters are placed perpendicular to each other on both sides. When no voltage is applied, light passes through (transparent). When voltage is applied, the liquid crystals align with the electric field, blocking light (dark/opaque).

In a dot-matrix LCD, each character position consists of a 5×8 or 5×10 matrix of pixels. The HD44780 controller drives these pixels based on ASCII codes sent by the microcontroller.

LCD Pin Configuration Summary

- Pins 1–2 (VSS, VCC): Power supply (GND and +5V)
- Pin 3 (VEE): Contrast control (0V = max contrast, typically via 10kΩ potentiometer)
- Pin 4 (RS): Register Select
- Pin 5 (RW): Read/Write control
- Pin 6 (E): Enable pulse
- Pins 7–14 (D0–D7): Data bus (8-bit or 4-bit mode using D4–D7)
- Pins 15–16 (LED+, LED-): Backlight LEDs

Function of RS (Register Select) Pin

The RS pin selects between two internal registers of the HD44780:

- **RS = 0 (Command Register):** The byte on D0–D7 is interpreted as an instruction/command. Examples: 0x38 (initialize 2-line, 8-bit mode), 0x01 (clear display), 0x80 (set cursor to first row, first column), 0xC0 (set cursor to second row).
- **RS = 1 (Data Register):** The byte on D0–D7 is interpreted as a character to be displayed. The ASCII code of the character is sent. Example: Sending 0x48 ('H') displays the letter H at the current cursor position.

Function of RW (Read/Write) Pin

- **RW = 0 (Write Mode):** Data/command is written to the LCD. This is the mode used in virtually all practical applications. The microcontroller sends data to the LCD.
- **RW = 1 (Read Mode):** Data is read from the LCD. Mainly used to check the Busy Flag (D7) — when D7=1, the LCD is busy processing a command. In most applications, RW is permanently connected to GND (always write mode), using time delays instead of checking the busy flag.

Function of EN (Enable) Pin

The Enable pin acts as a clock/latch signal for the LCD. Data on the D0–D7 lines is accepted by the HD44780 controller only on the falling edge (HIGH-to-LOW transition) of the EN pulse.

Sequence: Set RS, set RW, put data on D0–D7 → Pulse EN HIGH → wait 1 µs → pull EN LOW → wait for LCD to process (1–2 ms for commands, 1 ms for data).

Q14. Draw and explain the interfacing of a 16×2 LCD with a microcontroller in 4-bit mode.

Answer:

4-Bit Mode – Introduction

In normal 8-bit mode, the LCD requires 8 data lines (D0–D7) plus 3 control lines (RS, RW, EN) — a total of 11 GPIO pins. This is a significant use of GPIO pins on a microcontroller. 4-bit mode reduces

Available on notes.arpanadhikari7.com.np

this to only 4 data lines (D4–D7) plus 3 control lines, using just 7 GPIO pins. The data is transmitted in two 4-bit nibbles (upper nibble first, then lower nibble).

Hardware Connection (4-bit mode)

- LCD D4 → MCU PD4
- LCD D5 → MCU PD5
- LCD D6 → MCU PD6
- LCD D7 → MCU PD7
- LCD RS → MCU PC0
- LCD RW → MCU PC1 (or tied to GND)
- LCD EN → MCU PC2
- LCD VEE → Center of 10kΩ potentiometer (for contrast)
- LCD VCC → +5V; LCD VSS → GND

Initialization Sequence for 4-bit Mode

15. Power-on delay: Wait >15 ms after power up
16. Send 0x30 three times (with appropriate delays) to wake up in 8-bit mode
17. Send 0x20 to switch to 4-bit mode (only upper nibble used)
18. Send 0x28 (two nibbles: 0x2, then 0x8): Set 2-line, 5×8 character, 4-bit interface
19. Send 0x0C: Display ON, cursor OFF
20. Send 0x06: Auto-increment cursor
21. Send 0x01: Clear display (wait 1.64 ms)
22. Send 0x80: Set cursor to beginning of first row

Data Transmission in 4-bit Mode

For every byte (command or character), two write cycles are needed: First cycle: Upper nibble (bits 7–4) placed on D4–D7 → EN pulse. Second cycle: Lower nibble (bits 3–0) placed on D4–D7 → EN pulse.

Example: To send 0x38 (0011 1000): First nibble = 0011 (0x3) → D4=1, D5=1, D6=0, D7=0 → EN pulse. Second nibble = 1000 (0x8) → D4=0, D5=0, D6=0, D7=1 → EN pulse.

Block Diagram Description

[ATmega32] ─(PC0=RS, PC1=RW, PC2=EN)─→ [LCD Control Pins] and [ATmega32] ─(PD4–PD7=D4–D7)─→ [LCD Data Pins D4–D7] → [HD44780 Controller] → [LCD Display Matrix 16×2]

notes.arpanadhikari7.com.np

Advantages of 4-bit Mode

- Saves 4 GPIO pins compared to 8-bit mode
- Compatible with the same HD44780 commands
- Slightly more software complexity, but well-compensated by hardware savings
- Preferred for most embedded projects where GPIO pins are limited

Q15. What are the advantages of using LCD over LED displays?

Answer:

Introduction

Available on notes.arpanadhikari7.com.np

Both LCDs (Liquid Crystal Displays) and LED displays (7-segment or dot-matrix LED arrays) are used in embedded systems for displaying output. However, LCDs offer several significant advantages over simple LED displays, making them the preferred choice for modern embedded applications.

Advantages of LCD over LED Displays

- **1. Information Capacity:** An LCD 16×2 can display 32 alphanumeric characters simultaneously, including letters, numbers, and special symbols. A 7-segment LED display can only show one digit per module and has limited character representation (cannot display all letters like X, Z, K, etc.).
- **2. Versatility:** LCDs can display custom messages, sensor readings, status information, menus, and even custom characters. LED displays are restricted to numeric and a few hexadecimal characters.
- **3. Built-in Controller:** The LCD module contains an HD44780 controller IC that handles refresh cycles, cursor management, and character mapping. The CPU only needs to send ASCII codes. LED displays require continuous multiplexing by the CPU — consuming significant processing time.
- **4. Lower CPU Overhead:** Since the LCD has its own controller, the microcontroller is freed from managing individual LED segments. With LED displays, the CPU must continuously scan and update all digits, consuming processing cycles.
- **5. Low Power Consumption:** LCDs consume significantly less power than LED arrays because they do not emit light themselves (except the backlight). Multiple LED segments draw substantial current, especially in large displays.
- **6. Better Readability:** LCDs provide clear, character-based output that is easy for users to read. They support custom messages, improving user experience. LED displays showing scrolling or multiplexed digits can be harder to read.
- **7. Cost Effectiveness at Scale:** For displaying more than 4 digits, an LCD becomes more economical than multiple 7-segment LED display modules.
- **8. Easier Programming:** Displaying a string on LCD requires sending ASCII characters. Displaying the same on LED requires segment-code lookup tables and multiplexing logic.
- **9. Backlight Option:** LCDs have an optional LED backlight for visibility in dark environments, with a simple on/off or PWM-controlled brightness control.

Q16. Explain the construction and working of a seven-segment display.

Answer:

Introduction

A seven-segment display is an electronic display device used to represent decimal numerals (0–9) and some hexadecimal characters (A–F). It consists of seven LEDs (segments) arranged in a specific pattern that resembles the digit 8. By selectively illuminating certain segments, different characters can be displayed.

Construction

The display has seven LED segments labeled from 'a' to 'g', arranged as follows: Segment 'a' forms the top horizontal bar; segments 'b' and 'c' form the upper and lower right vertical bars; segments 'e' and 'f' form the lower and upper left vertical bars; segment 'd' forms the bottom horizontal bar; segment 'g' forms the middle horizontal bar. An optional decimal point (DP) is placed at the lower right.

Available on notes.arpanadhikari7.com.np

Physically, each segment is a small LED. All seven segment LEDs share either a common anode (CA) or common cathode (CC) connection. The segments are typically manufactured as a single chip with internal connections, packaged in a DIP (Dual Inline Package) housing.

Working Principle

By applying the correct logic levels to segments a–g, any decimal digit can be formed. For example: Digit '0' = segments a, b, c, d, e, f ON, g OFF. Digit '8' = all segments ON. Digit '1' = segments b, c ON only.

Segment Truth Table (Common Cathode – Logic 1 = ON)

- 0: a=1,b=1,c=1,d=1,e=1,f=1,g=0 → Hex 0x3F
- 1: a=0,b=1,c=1,d=0,e=0,f=0,g=0 → Hex 0x06
- 2: a=1,b=1,c=0,d=1,e=1,f=0,g=1 → Hex 0x5B
- 3: a=1,b=1,c=1,d=1,e=0,f=0,g=1 → Hex 0x4F
- 4: a=0,b=1,c=1,d=0,e=0,f=1,g=1 → Hex 0x66
- 5: a=1,b=0,c=1,d=1,e=0,f=1,g=1 → Hex 0x6D
- 6: a=1,b=0,c=1,d=1,e=1,f=1,g=1 → Hex 0x7D
- 7: a=1,b=1,c=1,d=0,e=0,f=0,g=0 → Hex 0x07
- 8: a=1,b=1,c=1,d=1,e=1,f=1,g=1 → Hex 0x7F
- 9: a=1,b=1,c=1,d=1,e=0,f=1,g=1 → Hex 0x6F

Multiplexing Multiple Displays

For displaying multi-digit numbers, multiple 7-segment displays are multiplexed. Each display's common terminal is connected to a transistor switch controlled by the MCU. The MCU rapidly switches between displays (refresh rate > 50 Hz) while putting the appropriate segment code on the shared data bus. Due to persistence of vision, all digits appear lit simultaneously.

Q17. Differentiate between common anode and common cathode seven-segment displays.

Answer:

Common Cathode (CC) Seven-Segment Display

notes.arpanadhikari7.com.np

In a common cathode display, all the cathode terminals (negative terminals) of the seven LED segments are connected together to form a single common terminal. This common terminal is connected to GND (logic 0). To illuminate a segment, a HIGH logic level (logic 1, +5V) is applied to its anode pin.

Logic: Segment ON = HIGH (1); Segment OFF = LOW (0). Example: To display '1', apply HIGH to segments b and c, LOW to all others.

Common Anode (CA) Seven-Segment Display

In a common anode display, all the anode terminals (positive terminals) of the seven LED segments are connected together. This common terminal is connected to VCC (+5V). To illuminate a segment, a LOW logic level (logic 0, GND) is applied to its cathode pin.

Logic: Segment ON = LOW (0); Segment OFF = HIGH (1). Example: To display '1', apply LOW to segments b and c, HIGH to all others. The segment codes are inverted compared to common cathode.

Key Differences

Available on notes.arpanadhikari7.com.np

- **Common Terminal:** CC → Common terminal connected to GND. CA → Common terminal connected to VCC.
- **Activation Logic:** CC → Segment lights up when pin is HIGH (active HIGH). CA → Segment lights up when pin is LOW (active LOW).
- **Segment Codes:** CC and CA use inverted segment codes. If CC code for '0' is 0x3F, CA code for '0' is 0xC0 (bitwise NOT).
- **Driving Circuit:** CC → Can be driven directly by MCU GPIO (current source). CA → Requires current sink (transistor or MCU pin as output LOW).
- **Microcontroller Interface:** CC is more naturally compatible with MCU GPIO pins that source current. CA requires the MCU to sink current or use a driver transistor.
- **Power Supply:** CC: Common pin to GND — MCU pins provide current. CA: Common pin to VCC — current flows through MCU pin to GND.
- **Applications:** Both are used equally in practice. CC displays are slightly more common in MCU-based projects. CA displays are common when using a higher voltage supply or when compatible driver ICs are used.

Q18. Write the hexadecimal codes required to display digits 0–9 on a common cathode seven-segment display.

Answer:

Seven-Segment Display Segment Mapping

In a common cathode (CC) display, a segment is ON when its pin is HIGH (logic 1). The 8-bit code represents segment pattern: bit7=DP, bit6=g, bit5=f, bit4=e, bit3=d, bit2=c, bit1=b, bit0=a. In most MCU connections, the bit order is: bit7=g, bit6=f, bit5=e, bit4=d, bit3=c, bit2=b, bit1=a, bit0=DP — but the standard convention used in code is: Byte = {DP, g, f, e, d, c, b, a}.

Standard Hex Codes for Digits 0–9 (Common Cathode, bit order: gfedcba)

- Digit 0: Segments ON: a,b,c,d,e,f; g=0 → Binary: 0 1 1 1 1 1 1 0 → Hexadecimal: 0x3F
- Digit 1: Segments ON: b,c only → Binary: 0 0 0 0 0 1 1 0 → Hexadecimal: 0x06
- Digit 2: Segments ON: a,b,d,e,g → Binary: 0 1 0 1 1 0 1 1 → Hexadecimal: 0x5B
- Digit 3: Segments ON: a,b,c,d,g → Binary: 0 1 0 0 1 1 1 1 → Hexadecimal: 0x4F
- Digit 4: Segments ON: b,c,f,g → Binary: 0 1 1 0 0 1 1 0 → Hexadecimal: 0x66
- Digit 5: Segments ON: a,c,d,f,g → Binary: 0 1 1 0 1 1 0 1 → Hexadecimal: 0x6D
- Digit 6: Segments ON: a,c,d,e,f,g → Binary: 0 1 1 1 1 1 0 1 → Hexadecimal: 0x7D
- Digit 7: Segments ON: a,b,c → Binary: 0 0 0 0 0 1 1 1 → Hexadecimal: 0x07
- Digit 8: Segments ON: ALL → Binary: 0 1 1 1 1 1 1 1 → Hexadecimal: 0x7F
- Digit 9: Segments ON: a,b,c,d,f,g → Binary: 0 1 1 0 1 1 1 1 → Hexadecimal: 0x6F

C Array Declaration for MCU Program

```
char seg_codes[] = {0x3F, 0x06, 0x5B, 0x4F, 0x66, 0x6D, 0x7D, 0x07, 0x7F, 0x6F};
```

For Common Anode display, use the bitwise NOT of each value:

```
char seg_codes_CA[] = {0xC0, 0xF9, 0xA4, 0xB0, 0x99, 0x92, 0x82, 0xF8, 0x80, 0x90};
```

Available on notes.arpanadhikari7.com.np

Q19. What is sensor interfacing, and why is it important in embedded systems? What are the primary steps involved in interfacing a sensor with a microcontroller?

Answer:

Sensor Interfacing – Definition

Sensor interfacing is the complete process of connecting a sensor device to a microcontroller, including hardware connections, signal conditioning, ADC configuration (for analog sensors), protocol implementation (for digital sensors), and software routines to read, process, and use the sensor's output data.

Importance of Sensor Interfacing

- Embedded systems cannot perceive the physical world without sensors. Proper interfacing ensures accurate, reliable data acquisition from the environment.
- Incorrect interfacing can lead to wrong readings, system malfunctions, or hardware damage.
- It enables automation: the system can sense, decide, and act without human intervention.
- Critical in safety systems (medical, automotive, industrial) where sensor accuracy is non-negotiable.

Primary Steps for Interfacing a Sensor with a Microcontroller

- **Step 1 – Select the Right Sensor:** Choose based on measurement range, accuracy, output type (analog/digital), power requirements, and communication protocol.
- **Step 2 – Understand the Sensor Datasheet:** Study pin configuration, supply voltage, output range, timing requirements, and communication protocol.
- **Step 3 – Hardware Connection:** Connect power (Vcc, GND), output pin to MCU GPIO or ADC pin. Add signal conditioning components (pull-up resistors, capacitors, op-amp).
- **Step 4 – Configure the Microcontroller:** Set GPIO direction (DDRA, DDRB), configure ADC registers (ADMUX, ADCSRA), or configure communication peripherals (UART, I2C, SPI).
- **Step 5 – Write the Driver Software:** Implement initialization routine, data reading functions, and data conversion formulas.
- **Step 6 – Data Processing and Calibration:** Apply scaling, offset correction, and calibration constants. Compare with reference measurements.
- **Step 7 – Test and Validate:** Verify sensor readings against known standards. Test across the full range and under different conditions.
- **Step 8 – Error Handling:** Implement timeout detection, checksum validation, and fallback behavior for sensor failures.

notes.arpanadhikari7.com.np

Q20. What are the factors to consider when selecting a servo motor for a specific application? Also mention typical steps for initializing and reading data from a sensor.

Answer:

Factors for Servo Motor Selection

- **1. Torque Rating:** The torque requirement depends on the load the servo must move. Servo torque is rated in kg-cm or oz-in (e.g., MG996R: 9.4 kg-cm at 4.8V). Always select a servo with torque at least 1.5× the required load torque for safety margin.

Available on notes.arpanadhikari7.com.np

- **2. Speed:** Servo speed is rated in seconds/60° (e.g., 0.2 sec/60° at 4.8V). For fast-response applications like camera gimbals, select faster servos. For precision positioning where speed is less critical, slower servos are acceptable.
- **3. Rotation Range:** Standard servos rotate 0°–180°. Continuous rotation servos do 360°. Select based on the required range of motion (e.g., door lock mechanism needs only 90°, robotic arm joint may need 180°).
- **4. Size and Weight:** In weight-sensitive applications (drones, small robots), use nano or micro servos. For industrial applications, standard or high-torque servos are appropriate.
- **5. Gear Type:** Plastic gears: lightweight, suitable for light loads. Metal gears: more durable, for higher torque and repeated cycles. Titanium gears: highest durability, used in critical applications.
- **6. Power Supply Voltage:** Most servo motors operate at 4.8–6V. Some industrial servos work at 12V or 24V. Ensure the system power supply matches.
- **7. Feedback Mechanism:** Analog servo: uses potentiometer. Digital servo: uses encoder for higher accuracy and faster response.
- **8. Environment:** Waterproof/dustproof servos (IP rating) for outdoor or harsh conditions.

Typical Steps for Initializing and Reading Data from a Sensor

23. Power up the microcontroller and wait for supply voltage to stabilize (>15 ms for LCD, >250 ms for some sensors).
24. Configure GPIO: Set DDRA/DDRB/DDRC/DDRD registers — input pins to 0, output pins to 1.
25. Initialize ADC: Set ADMUX (reference voltage, input channel), set ADCSRA (enable ADC, set prescaler).
26. For digital sensors: Initialize the communication protocol (I2C: TWBR register; UART: UBRRH/UBRRL, UCSRB, UCSRC; SPI: SPCR, SPSR).
27. Send initialization command to the sensor if required (e.g., DHT11 start signal, LCD initialization sequence).
28. Wait for the sensor to complete its initialization/startup sequence.
29. Start data acquisition: For ADC, set ADSC=1; for digital sensors, trigger data request.
30. Wait for conversion/data to complete: Poll ADIF flag or receive interrupt.
31. Read data registers: ADCL then ADCH for ADC; receive bytes for digital sensors.
32. Apply calibration and conversion formula to get meaningful physical units.

notes.arpanadhikari7.com.np

Q27. What are the primary types of motors used in embedded systems, and what are their key differences?

Answer:

Types of Motors in Embedded Systems

Motors are the primary actuators in embedded systems, converting electrical energy into mechanical motion. The three main types are DC Motors, Stepper Motors, and Servo Motors. Each has distinct characteristics suited for different applications.

1. DC (Direct Current) Motor

DC motors produce continuous rotational motion when a DC voltage is applied. Speed is proportional to voltage. Direction is controlled by reversing polarity. Motor driver ICs (L293D, L298N) are used since MCU GPIO pins cannot supply sufficient current. Speed control uses PWM.

Available on notes.arpanadhikari7.com.np

Key Features: Simple, inexpensive, high RPM, suitable for continuous rotation. Used in: fans, wheels, drills, conveyor belts.

2. Stepper Motor

Stepper motors move in discrete angular steps. Each electrical pulse advances the rotor by a fixed angle (e.g., 1.8°). No feedback sensor required — open-loop position control. Driving modes: Wave Drive (1 coil), Full Step (2 coils), Half Step (alternating). Used in CNC, 3D printers, and precision instruments.

Key Features: High positional accuracy, no feedback needed, can hold position. Used in: plotters, camera systems, hard drives.

3. Servo Motor

Servo motors use closed-loop feedback control (potentiometer or encoder) for precise angular positioning. Controlled by PWM signal (1–2 ms pulse, 50 Hz). Range: typically 0°–180°. Used in robotics, RC vehicles, automated systems requiring precise position control.

Key Features: Precise position control, closed-loop, handles varying loads. Used in: robotic arms, camera gimbals, industrial automation.

Key Differences Summary

- **Control Type:** DC Motor = Open-loop speed; Stepper = Open-loop position; Servo = Closed-loop position
- **Precision:** DC < Stepper < Servo
- **Feedback:** DC and Stepper = None required; Servo = Potentiometer/Encoder
- **Control Signal:** DC = Voltage/PWM; Stepper = Digital pulses; Servo = PWM (1–2 ms)
- **Cost:** DC (cheapest) < Stepper < Servo (most expensive)

Q31. You are designing a weather monitoring system using analog sensors. Explain how ADC selection affects system performance.

Answer:

Weather Monitoring System Overview

A weather monitoring system typically uses multiple analog sensors: LM35 (temperature), capacitive humidity sensor, BMP180 (pressure via ADC), LDR (light/UV), and anemometer (wind speed via analog voltage). All these sensors produce analog voltages that must be converted by ADC for the microcontroller to process.

How ADC Selection Affects System Performance

- **1. Resolution and Measurement Accuracy:** The ADC resolution determines how finely the system can distinguish between sensor readings. An 8-bit ADC (256 levels) with a 5V reference gives a step size of 5/256 ≈ 19.5 mV. If the LM35 outputs 10 mV/°C, the system can only resolve temperature in ~2°C increments — unacceptable for a weather station. A 10-bit ADC (1024 levels) with 2.56V reference gives 2.5 mV/step, resolving 0.25°C increments. A 12-bit ADC gives 0.061°C resolution. For a weather station requiring ±0.1°C accuracy, a 12-bit or higher ADC is needed.
- **2. Sampling Rate (Conversion Speed):** Fast-changing parameters (wind speed, rain gauge) need rapid sampling. Slow ADC conversion time can miss peaks. ATmega32's ADC at CK/128 with 8 MHz clock gives ~76,923 samples/second — sufficient for most weather monitoring but

Available on notes.arpanadhikari7.com.np

may be inadequate for capturing sudden wind gusts if the MCU is also busy with display and communication tasks.

- **3. Reference Voltage Stability:** The accuracy of the ADC depends critically on the stability of Vref. If Vref fluctuates (due to noisy power supply), the ADC readings will be inconsistent — critical for a weather station where readings are logged over time. Using the ATmega32's stable internal 2.56V reference or a precision external reference (e.g., LM4040) significantly improves measurement consistency.
- **4. Number of ADC Channels:** A weather station may have 5+ analog sensors. ATmega32 has 8 ADC channels (PA0–PA7), which is sufficient. However, if more sensors are needed, an external multiplexer (e.g., CD4051) or a higher-channel ADC IC must be used. Channel cross-talk can affect accuracy — dedicated ADC ICs are preferable for high-precision systems.
- **5. ADC Input Impedance:** High-impedance analog sensors (e.g., capacitive humidity sensors) can cause slow charging of the ADC's internal sample-and-hold capacitor, leading to errors if conversion starts too quickly. A longer acquisition time setting (ADSC trigger delay) or an op-amp buffer can solve this.
- **6. Noise and Signal Conditioning:** Analog signals in a weather station environment can pick up electrical noise from motors, radio interference, etc. Using hardware averaging (multiple ADC readings averaged in software), oversampling, or adding RC filters before the ADC input improves accuracy.

Q41. How does varying the duty cycle of PWM affect the speed and torque of a DC motor?

Answer:

PWM and DC Motor Relationship

The speed and torque characteristics of a DC motor are directly related to the average voltage and current applied to it. PWM (Pulse Width Modulation) provides a digitally controlled method of varying the average voltage delivered to the motor by varying the duty cycle.

Effect of Duty Cycle on Motor Speed

The average voltage delivered by PWM is: $V_{avg} = Vcc \times \text{Duty Cycle}$. A DC motor's speed (in RPM) is approximately proportional to the back-EMF, which is proportional to the average terminal voltage. Therefore: Speed $\propto V_{avg} \propto$ Duty Cycle.

notes.arpanadhikari7.com.np

- 0% duty cycle: $V_{avg} = 0V \rightarrow$ Motor stopped
- 25% duty cycle: $V_{avg} = 1.25V \rightarrow$ ~25% of full speed
- 50% duty cycle: $V_{avg} = 2.5V \rightarrow$ ~50% of full speed
- 75% duty cycle: $V_{avg} = 3.75V \rightarrow$ ~75% of full speed
- 100% duty cycle: $V_{avg} = 5V \rightarrow$ Full speed

Effect of Duty Cycle on Motor Torque

Torque in a DC motor is proportional to current: $T = K \times I$, where K is the motor's torque constant. The current depends on the average voltage and the motor's resistance. As duty cycle increases, average voltage increases, which increases current (and thus torque), assuming the motor is not in saturation.

However, at very low duty cycles, the motor may not develop enough torque to overcome static friction and stall. The minimum duty cycle to start the motor is called the dead band. Below this point, the motor doesn't rotate even though some power is delivered.

Available on notes.arpanadhikari7.com.np

PWM Frequency Considerations

PWM frequency affects the smoothness of motor operation. At low PWM frequencies (< 1 kHz): motor may emit audible noise (whining); current ripple is higher; speed control is less smooth. At high PWM frequencies (> 5 kHz): smoother operation; reduced audible noise; motor inductance naturally filters the current ripple. For DC motor control, 5 kHz–20 kHz is the recommended PWM frequency range.

Practical Implementation

On ATmega32: Configure Timer0 or Timer1 in Fast PWM mode. Connect PWM output to Enable pin of L293D motor driver. Change OCR value in software to vary speed: OCR = 0 (stop), OCR = 127 (50%), OCR = 255 (full speed). Use IN1/IN2 pins of L293D for direction control.

Q46. How does a servo motor use PWM for position control, and what is the relationship between the PWM duty cycle and the servo's angular position?

Answer:

Servo Motor PWM Position Control

Unlike DC motors where PWM duty cycle controls speed, for servo motors, the PWM pulse width (not percentage duty cycle) directly determines the angular position of the shaft. The servo's internal comparator circuit interprets the incoming pulse width as a position command.

PWM Signal Specifications for Servo Motors

- Signal frequency: 50 Hz (period = 20 ms)
- Pulse width range: 1.0 ms (0°) to 2.0 ms (180°)
- Neutral position pulse: 1.5 ms (90°)
- Signal voltage: TTL compatible (0V / 5V)

Position to Pulse Width Relationship

The relationship is linear: Pulse Width (ms) = 1.0 + (angle / 180°) × 1.0. Thus: 0° → 1.0 ms; 45° → 1.25 ms; 90° → 1.5 ms; 135° → 1.75 ms; 180° → 2.0 ms.

Duty Cycle to Position Relationship (for 20ms period)

Duty Cycle = Pulse Width / Period × 100%. For 0°: 1.0/20 × 100 = 5%. For 90°: 1.5/20 × 100 = 7.5%. For 180°: 2.0/20 × 100 = 10%. So the useful duty cycle range for servo control is only 5% to 10%.

notes.arpanadhikari7.com.np

Closed-Loop Control Mechanism

33. The microcontroller generates a PWM pulse of the desired width (e.g., 1.5 ms for 90°).
34. The servo's internal comparator measures the incoming pulse width.
35. The comparator reads the potentiometer voltage — proportional to current shaft position.
36. If the potentiometer voltage does not match the commanded position, the error signal drives the motor.
37. The motor rotates until the potentiometer voltage matches the command — at which point the error = 0 and the motor stops.
38. The process repeats with each incoming pulse (every 20 ms), continuously correcting any positional drift.

Available on notes.arpanadhikari7.com.np

SCENARIO-BASED QUESTIONS

Scenario 1: Smart Room Temperature Control

A smart room uses a temperature sensor, a DC fan, and a 16×2 LCD to maintain a comfortable temperature.

QS1.1. Identify the type of temperature sensor suitable for this system and justify your choice.

Answer:

The LM35 temperature sensor is most suitable for this smart room temperature control system.

Justification

- **Linear Analog Output:** LM35 provides a linear analog output of 10 mV/°C, making it directly compatible with the ATmega32's internal 10-bit ADC without external signal conditioning.
- **Calibrated in Celsius:** Since room temperature monitoring is done in Celsius, LM35's direct Celsius scale output simplifies software calculation (Temperature = ADC/4 when Vref = 2.56V).
- **Operating Range:** LM35 operates from -55°C to +150°C, well beyond typical room temperature ranges (15°C–40°C), ensuring reliable operation.
- **Accuracy:** ±0.5°C accuracy is sufficient for room temperature control (setpoint typically ±1°C tolerance).
- **Simple Interfacing:** 3-pin sensor (Vcc, Vout, GND) — directly connected to ADC input PA0. No external components needed.
- **Alternatively:** DHT11 can be used if humidity monitoring is also required. It provides both temperature and humidity digitally over a single wire.

QS1.2. Explain how the sensor data is processed before controlling the fan.

Answer:

39. LM35 Vout connected to ADC channel PA0 (ATmega32).
40. ADMUX = 0xC0 (Vref=2.56V, right-justified, ADC0 channel).
41. ADCSRA = 0x87 (ADC enable, prescaler CK/128).
42. Start conversion: ADCSRA |= (1 << ADSC).
43. Wait for ADIF flag: while (!(ADCSRA & (1<<ADIF)));
44. Read ADC result: value = ADCL | (ADCH << 8).
45. Convert to Celsius: temperature = value / 4; (since LM35 output=10mV/°C and step size=2.5mV, 10/2.5=4).
46. Compare temperature with thresholds: if temp > 28°C, activate fan; if temp > 32°C, increase fan speed; if temp < 25°C, turn off fan.

QS1.3. Analyze how PWM is used to control the fan speed.

Answer:

The DC fan's speed is controlled by PWM (Pulse Width Modulation) through a motor driver IC (L293D or ULN2003).

Available on notes.arpanadhikari7.com.np

- **25°C–28°C (Comfortable):** Fan OFF: PWM duty cycle = 0% (OCR = 0)
- **28°C–30°C (Warm):** Fan LOW speed: PWM duty cycle = 30% (OCR = 77 for 8-bit timer)
- **30°C–33°C (Hot):** Fan MEDIUM speed: PWM duty cycle = 60% (OCR = 153)
- **Above 33°C (Very Hot):** Fan FULL speed: PWM duty cycle = 100% (OCR = 255)

Timer0 of ATmega32 configured in Fast PWM mode 3 (TCCR0 = 0x67). OCR0 register value is changed based on temperature reading. Higher duty cycle → higher average voltage → faster fan rotation.

QS1.4. Explain how the current temperature and fan status can be displayed on the LCD.

Answer:

47. Initialize LCD: LCD_Init() sets 8-bit or 4-bit mode, clears display, sets cursor.
48. First row (0x80): Display temperature — LCD_Command(0x80); LCD_String("Temp: "); convert integer to string using itoa(); LCD_String(temp_str); LCD_String(" C");
49. Second row (0xC0): Display fan status — LCD_Command(0xC0); if (fan_speed==0) LCD_String("Fan: OFF "); else LCD_String("Fan: ON "+speed_str+"%");
50. Update display every 500 ms using a delay loop or Timer interrupt.
51. Use LCD_Command(0x01) to clear display before updating to prevent character ghosting.

QS1.5. What problems may arise if the sensor is not calibrated properly?

Answer:

- **Incorrect Temperature Readings:** An uncalibrated sensor may read 30°C when actual temperature is 25°C, causing the fan to activate unnecessarily, wasting energy and causing discomfort.
- **Wrong Fan Activation Threshold:** If the sensor has a systematic offset (e.g., always reading 3°C higher), the fan activates at 25°C instead of 28°C.
- **Non-Linear Errors:** If the sensor's output is not perfectly linear due to manufacturing variations, readings may be accurate at calibration point but drift at other temperatures.
- **ADC Reference Voltage Errors:** If Vref is unstable, the ADC conversion formula (T = ADC/4) gives incorrect results.
- **Display Discrepancy:** Users see incorrect temperature on LCD, reducing system credibility.

notes.arpanadhikari7.com.np

PROGRAM QUESTIONS

QP1. Write an AVR C program to interface a 7-segment display. Display numbers from 0 to 9 in a loop with a delay of 1 second.

Answer:

Circuit: ATmega32 Port C (PC0–PC6) connected to segments a–g of Common Cathode 7-segment display. Resistors of 330Ω in series with each segment.

```
#define F_CPU 8000000UL
```


Available on notes.arpanadhikari7.com.np

```
#include <avr/io.h>
#include <util/delay.h>

// Segment codes for digits 0-9 (Common Cathode)
// Bit order: bit6=g, bit5=f, bit4=e, bit3=d, bit2=c, bit1=b, bit0=a
const uint8_t seg_code[] = {
    0x3F, // 0: a,b,c,d,e,f ON (g OFF)
    0x06, // 1: b,c ON
    0x5B, // 2: a,b,d,e,g ON
    0x4F, // 3: a,b,c,d,g ON
    0x66, // 4: b,c,f,g ON
    0x6D, // 5: a,c,d,f,g ON
    0x7D, // 6: a,c,d,e,f,g ON
    0x07, // 7: a,b,c ON
    0x7F, // 8: all ON
    0x6F  // 9: a,b,c,d,f,g ON
};

int main(void) {
    DDRC = 0xFF;   // Set Port C as output
    PORTC = 0x00;  // All segments OFF initially

    while(1) {
        for (int i = 0; i < 10; i++) {
            PORTC = seg_code[i];  // Output segment code
            _delay_ms(1000);       // 1 second delay
        }
    }
    return 0;
}
```

QP2. Write an AVR C program to display your name on the first row and 'Hello!' on the second row of a 16x2 LCD.

Answer:

notes.arpanadhikari7.com.np

Hardware: LCD data bus on Port D (PD0–PD7), RS→PC0, RW→PC1, EN→PC2. 8-bit mode.

```
#define F_CPU 8000000UL
#include <avr/io.h>
#include <util/delay.h>

#define LCD_DATA  PORTD
#define LCD_CTRL  PORTC
#define LCD_DDR_D DDRD
#define LCD_DDR_C DDRC
#define RS PC0
#define RW PC1
#define EN PC2

void LCD_EnablePulse(void) {
```


Available on notes.arpanadhikari7.com.np

```
    LCD_CTRL |= (1<<EN);
    _delay_us(1);
    LCD_CTRL &= ~(1<<EN);
    _delay_ms(2);
}

void LCD_Command(uint8_t cmd) {
    LCD_DATA = cmd;
    LCD_CTRL &= ~(1<<RS);  // RS=0: Command
    LCD_CTRL &= ~(1<<RW);  // RW=0: Write
    LCD_EnablePulse();
}

void LCD_Data(uint8_t data) {
    LCD_DATA = data;
    LCD_CTRL |= (1<<RS);   // RS=1: Data
    LCD_CTRL &= ~(1<<RW);  // RW=0: Write
    LCD_EnablePulse();
}

void LCD_Init(void) {
    LCD_DDR_D = 0xFF;
    LCD_DDR_C = 0xFF;
    _delay_ms(20);
    LCD_Command(0x38); // 8-bit, 2-line, 5x8 font
    LCD_Command(0x0C); // Display ON, cursor OFF
    LCD_Command(0x06); // Auto-increment cursor
    LCD_Command(0x01); // Clear display
    LCD_Command(0x80); // Cursor to beginning row 1
}

void LCD_Print(const char *str) {
    while (*str) {
        LCD_Data(*str++);
    }
}

int main(void) {
    LCD_Init();
    LCD_Command(0x80);        // Row 1, Col 1
    LCD_Print("   Your Name   "); // Row 1: Display name
    LCD_Command(0xC0);        // Row 2, Col 1
    LCD_Print("    Hello!     "); // Row 2: Hello!
    while(1) {}  // Stay on display
    return 0;
}
```

notes.arpanadhikari7.com.np

QP3. Write an AVR C program to display the message 'Hello, World!' on a 16x2 LCD using 4-bit mode.

Available on notes.arpanadhikari7.com.np

Answer:

Hardware: LCD D4–D7 on PD4–PD7, RS→PC0, RW→PC1, EN→PC2. 4-bit mode.

```
#define F_CPU 8000000UL
#include <avr/io.h>
#include <util/delay.h>

#define LCD_DATA  PORTD
#define LCD_CTRL  PORTC
#define RS PC0
#define RW PC1
#define EN PC2

void LCD_Pulse(void) {
    LCD_CTRL |= (1<<EN);
    _delay_us(1);
    LCD_CTRL &= ~(1<<EN);
    _delay_ms(2);
}

void LCD_SendNibble(uint8_t nibble) {
    LCD_DATA = (LCD_DATA & 0x0F) | (nibble & 0xF0);
    LCD_Pulse();
}

void LCD_Command4(uint8_t cmd) {
    LCD_CTRL &= ~(1<<RS);
    LCD_CTRL &= ~(1<<RW);
    LCD_SendNibble(cmd & 0xF0);        // High nibble
    LCD_SendNibble((cmd << 4) & 0xF0); // Low nibble
}

void LCD_Data4(uint8_t data) {
    LCD_CTRL |= (1<<RS);
    LCD_CTRL &= ~(1<<RW);
    LCD_SendNibble(data & 0xF0);        // High nibble
    LCD_SendNibble((data << 4) & 0xF0); // Low nibble
}

void LCD_Init4(void) {
    DDRD = 0xFF; DDRC = 0xFF;
    _delay_ms(20);
    // Switch to 4-bit mode
    LCD_CTRL &= ~(1<<RS); LCD_CTRL &= ~(1<<RW);
    LCD_DATA = 0x30; LCD_Pulse(); _delay_ms(5);
    LCD_DATA = 0x30; LCD_Pulse(); _delay_ms(1);
    LCD_DATA = 0x30; LCD_Pulse(); _delay_ms(1);
    LCD_DATA = 0x20; LCD_Pulse(); _delay_ms(1); // 4-bit mode
    LCD_Command4(0x28); // 4-bit, 2-line, 5x8
    LCD_Command4(0x0C); // Display ON, Cursor OFF
    LCD_Command4(0x06); // Auto-increment
    LCD_Command4(0x01); // Clear display
```


Available on notes.arpanadhikari7.com.np

```
    _delay_ms(2);
}

void LCD_Print4(const char *str) {
    while (*str) LCD_Data4(*str++);
}

int main(void) {
    LCD_Init4();
    LCD_Command4(0x80);       // Row 1
    LCD_Print4("Hello, World!");
    LCD_Command4(0xC0);       // Row 2
    LCD_Print4("  AVR Project  ");
    while(1) {}
    return 0;
}
```

— End of Assignment 6 Answers —

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np